

Testing and Deployment Report

1. Scope

This report covers the STEMM Lab mobile app built for CSE3MAD Assessment 4. The testing strategy checks domain calculations, workflow readiness, activity catalogue completeness, sprint documentation, evidence capture, and deployment readiness.

2. Test Environment

- Framework: Expo / React Native.
- Language: TypeScript.
- Local database: Expo SQLite.
- Cloud adapter: Firestore REST API with demo-mode fallback.
- Device APIs: Expo Location, Sensors, Battery, Notifications, Task Manager, Background Task.
- Test runner: Jest with ts-jest.

Recommended physical test devices:

- Android phone for accelerometer/gyroscope/GPS/battery/notification evidence.
- Firebase Test Lab Android devices for APK smoke tests.

3. Automated Test Results

Run:

```
npm run verify
```

Implemented automated tests:

Test suite | Type | What it proves

`calculations.test.ts` | Unit | Drop velocity, acceleration, g-force, sound risk, motion stability, measurement completion, leaderboard scoring.

`readiness.test.ts` | Integration | Team profile, activities, sprint evidence, attempts, sensors, GPS/battery, Firestore demo sync, and evidence URI combine into rubric readiness.

`workflow.test.ts` | End-to-end style | The activity catalogue contains all seven activities; sprint records cover all three required sprints; team IDs/discriminators work for export.

Current local verification at generation time:

- TypeScript: passing.
- Jest: 3 test suites passing.
- Jest test count: 11 tests passing.

4. Manual End-to-End Test Script

1. Open the app.
2. Create team profile with team name, member names, and year level.
3. Confirm dashboard shows team discriminator.

4. Open Parachute Drop Challenge.
5. Record prediction, outcome, unit, notes, rating, and evidence URI.
6. Sample sensors/GPS/battery.
7. Save locally and sync.
8. Open Evidence screen.
9. Confirm attempt appears with member name, sync status, battery, sensor samples, and evidence URI.
10. Export JSON.
11. Open Testing screen and explain automated tests.
12. Open Sprint screen and explain sprint evidence.

Expected result: the app records a complete attempt, stores it locally, produces a sync status, updates readiness, and displays evidence for presentation.

5. Firebase Test Lab Plan

Firebase Test Lab requires an APK and Firebase console access. After building the APK:

1. Open Firebase Console.
2. Select the STEMM Lab project.
3. Go to Test Lab.
4. Upload the APK.
5. Select at least one Android device per team member.
6. Run a Robo test or instrumentation smoke test.
7. Save screenshots of:
 - Device matrix.
 - Passed/failed result summary.
 - Logs.
 - Screenshots/videos produced by Test Lab.

Each student should include a different Test Lab device result in the evidence folder.

6. Deployment Steps

Development:

```
npm install --legacy-peer-deps
npm start
```

Verification:

```
npm run verify
npm run test:report
```

APK build:

```
npm install -g eas-cli
eas login
eas build:configure
eas build -p android --profile preview
```

Submission packaging:

7. Issues Found and Improvements

During local implementation, TypeScript caught an Expo background task executor return type mismatch. The task was updated to return a Promise-compatible result. This improved reliability and demonstrates why static type checking was included.

8. Limitations

- Firebase Test Lab evidence cannot be generated without Firebase project access and an uploaded APK.
- Real GPS, accelerometer, gyroscope, camera/video, and notification evidence should be captured on a physical phone.
- AdMob production integration requires a production AdMob account and native build configuration. The project includes a test placement and official test banner ID for assessment demonstration.
- Web preview is useful for layout checks but cannot fully replace device testing.

9. Reflection

The strongest testing evidence combines automated tests with device evidence. Automated tests show that calculations, readiness scoring, catalogue integrity, and workflow logic work consistently. Manual and Firebase Test Lab tests prove the actual mobile build works on real devices with permissions, sensors, GPS, background tasks, and notifications.